

## Auto-test — servage de LLM sous charge

Vingt questions sur le terrain technique de la mission. Répondez d'abord, les corrigés sont en fin de document. L'objectif n'est pas d'avoir tout juste : c'est de repérer avant lundi les deux ou trois sujets où une question du client vous mettrait en difficulté.

### Servage et débit

1. Qu'est-ce que le *continuous batching* et pourquoi change-t-il davantage le débit que le batching statique ?
2. Que désignent TTFT et TPOT, et lequel dégrade le plus l'expérience perçue dans un assistant conversationnel ?
3. Qu'est-ce que PagedAttention et quel problème résout-il ?
4. Un GPU affiche 95 % d'utilisation. Pourquoi cela ne signifie-t-il pas qu'il est bien exploité ?
5. Deux serveurs d'inférence open source courants pour du Mistral on-premise, et leur différence principale ?

### Modèles et mémoire

6. Combien de VRAM faut-il approximativement pour servir un modèle de 7 milliards de paramètres en FP16 ? Et en INT4 ?
7. Différence entre AWQ, GPTQ et GGUF ? Lequel pour du GPU serveur ?
8. Qu'est-ce que le cache KV, comment croît-il, et pourquoi devient-il le premier facteur limitant en concurrence élevée ?
9. Parallélisme tensoriel et parallélisme de pipeline : lequel choisir quand le modèle ne tient pas sur un GPU, et quel est le coût de chacun ?
10. Qu'est-ce que le *speculative decoding* et quel gain en attendre ?

### RAG

11. Trois leviers pour réduire le nombre de tokens envoyés au modèle sans dégrader la qualité des réponses ?
12. À quoi sert un reranker, et pourquoi le placer après la recherche vectorielle plutôt que de l'y substituer ?
13. Que mesurent Hit Rate, MRR et nDCG, et lequel privilégier quand seule compte la présence du bon passage dans le contexte ?
14. Recherche hybride : que combine-t-elle, et dans quel cas apporte-t-elle le plus ?

**15.** Qu'est-ce qu'un cache sémantique, et quel est son risque principal ?

## Agents

**16.** Pourquoi un agent mal cadré peut-il multiplier la charge par cinq ou dix, et quels garde-fous poser ?

**17.** Que résout LangGraph par rapport à une chaîne LangChain classique ?

**18.** Qu'est-ce que le protocole MCP et quel problème d'intégration adresse-t-il ?

## Gouvernance

**19.** Quels critères font qu'un système d'IA relève du haut risque au sens de l'AI Act, et quelles obligations en découlent ?

**20.** Comment rendre une solution d'IA générative *réversible*, et pourquoi est-ce une exigence structurante chez un opérateur d'infrastructure ?

---

## Corrigés

**1.** Le batching statique attend de constituer un lot puis le traite jusqu'au bout : les requêtes courtes attendent les longues. Le continuous batching insère et retire des requêtes à chaque itération de génération, ce qui maintient le GPU plein. Gain de débit typique d'un facteur 2 à 4 sans changer le matériel.

**2.** *Time To First Token* : délai avant le premier mot. *Time Per Output Token* : cadence de génération ensuite. Le TTFT dégrade le plus la perception — au-delà d'une à deux secondes l'utilisateur croit que rien ne se passe, alors qu'un TPOT un peu lent reste tolérable une fois le flux démarré.

**3.** Une gestion paginée du cache KV, par blocs non contigus, à la manière de la mémoire virtuelle. Elle supprime la fragmentation et la sur-réservation liées à l'allocation d'un bloc contigu dimensionné sur la longueur maximale, et permet de servir bien plus de requêtes concurrentes à VRAM constante.

**4.** L'utilisation mesure la proportion de temps où au moins un noyau tourne, pas l'intensité du calcul. Un GPU passant son temps sur des transferts mémoire ou de petits noyaux affiche 95 % avec un rendement réel très bas. Les indicateurs utiles sont le MFU, le débit en tokens par seconde et l'occupation du cache KV.

**5.** vLLM et TGI (Text Generation Inference). vLLM est né du PagedAttention et excelle en débit sous forte concurrence ; TGI, de Hugging Face, est plus intégré à leur écosystème. SGLang et TensorRT-LLM sont les autres candidats sérieux, ce dernier étant le plus rapide mais le plus contraignant à exploiter.

6. Environ 14 Go pour les poids en FP16 (2 octets par paramètre), auxquels s'ajoutent le cache KV et l'espace d'activation — compter 18 à 20 Go en pratique. En INT4, environ 3,5 à 4 Go de poids, soit une dizaine de gigaoctets en tout. C'est ce qui fait passer un modèle d'un A100 à une carte grand public.

7. AWQ et GPTQ quantifient pour l'inférence GPU — AWQ préserve les poids saillants et dégrade généralement moins, GPTQ est plus ancien et très répandu. GGUF est le format de llama.cpp, orienté CPU et Apple Silicon. **Pour du GPU serveur : AWQ**, ou GPTQ selon le support du moteur retenu.

8. Le cache KV mémorise les clés et valeurs d'attention des tokens déjà traités, pour éviter de les recalculer. Il croît **linéairement avec la longueur de contexte et avec le nombre de requêtes simultanées**. À forte concurrence il dépasse la taille des poids et devient le vrai plafond : c'est lui qui limite le nombre d'utilisateurs, pas le modèle.

9. Le parallélisme tensoriel découpe chaque couche entre GPU : faible latence, mais exige une interconnexion rapide (NVLink) car il communique à chaque couche. Le pipeline découpe le modèle en tranches successives : communication réduite, mais bulles d'inactivité et latence accrue. **Sur un nœud, tensoriel ; entre nœuds, pipeline.**

10. Un petit modèle propose plusieurs tokens d'avance, le grand modèle les valide en une passe. Les propositions justes sont acceptées en bloc. Gain typique d'un facteur 1,5 à 3 sur la latence, sans changer la sortie — la validation garantit une distribution identique.

11. Découpage plus fin avec reranking pour ne garder que les passages utiles ; réduction du nombre de passages injectés (souvent 3 à 5 suffisent là où l'on en met 10) ; compression ou résumé du contexte. À quoi s'ajoute le cache sémantique sur les questions récurrentes.

12. La recherche vectorielle est rapide mais approximative ; le reranker est un modèle d'encodage croisé, précis mais coûteux. On enchaîne : récupérer largement, puis reclasser finement une vingtaine de candidats. L'utiliser seul serait inabordable, l'omettre laisse du bruit dans le contexte.

13. Le *Hit Rate* mesure la présence du bon passage dans les k premiers résultats ; le MRR, le rang du premier résultat pertinent ; le nDCG, la qualité de l'ordonnancement complet avec pondération par position. **Si seule compte la présence dans le contexte, c'est le Hit Rate** — le modèle génératif se charge du reste.

14. Recherche lexicale (BM25) et recherche vectorielle. La première excelle sur les termes exacts — références, codes, acronymes, noms propres — là où les embeddings les diluent. C'est déterminant sur du corpus technique et réglementaire, typiquement chez un opérateur de réseau.

15. Un cache indexé sur la proximité sémantique de la question plutôt que sur son texte exact. Risque principal : **servir une réponse périmée ou approchante** quand deux questions proches appellent des réponses différentes. Il faut un seuil de similarité prudent, une durée de vie courte, et une invalidation liée à la mise à jour du corpus.

**16.** Chaque étape d'un agent est un appel au modèle : boucles de raisonnement, appels d'outils, tentatives après échec se multiplient. Un cycle non borné peut tourner indéfiniment. Garde-fous : plafond d'itérations, budget de tokens par exécution, délai de garde, plafond d'appels d'outils, et arrêt sur absence de progression.

**17.** LangChain enchaîne linéairement. LangGraph modélise le flux comme un **graphe d'états** avec cycles, branchements conditionnels, reprise et point de contrôle. Cela rend gouvernables les agents qui doivent revenir en arrière, attendre une validation humaine ou reprendre après incident.

**18.** *Model Context Protocol* : un protocole ouvert normalisant la façon dont un modèle accède à des outils, des données et des invites. Il résout la prolifération des intégrations sur mesure — au lieu de N connecteurs par N clients, chaque source expose une fois une interface standard.

**19.** Notamment les systèmes touchant aux infrastructures critiques, à l'emploi, au crédit, à l'éducation, à la justice et aux services essentiels. Obligations : système de gestion des risques, gouvernance et qualité des données, documentation technique, journalisation, transparence vis-à-vis des utilisateurs, supervision humaine, robustesse et cybersécurité, plus une évaluation de conformité. **Le pilotage d'un réseau électrique relève de l'infrastructure critique** — le sujet n'est pas théorique ici.

**20.** Découplage strict du modèle : passer par une couche d'abstraction plutôt que par l'API d'un fournisseur, garder prompts, index et embeddings dans un format portable, documenter les évaluations pour pouvoir comparer un modèle de remplacement, et éviter les fonctions propriétaires sans équivalent. Chez un opérateur d'infrastructure, la durée de vie des systèmes se compte en décennies et dépasse de loin celle d'un fournisseur de modèle — d'où l'exigence, qui figure d'ailleurs explicitement dans les livrables attendus.

---

## Autodiagnostic

- **17 à 20** — vous tenez la conversation technique avec l'équipe Enedis.
- **13 à 16** — solide. Revoyez les deux ou trois sujets manqués avant lundi.
- **9 à 12** — les fondamentaux sont là, l'exploitation sous charge moins. Concentrez-vous sur les questions 1 à 10, c'est le cœur de la mission.
- **moins de 9** — le sujet réel est le servage sous contrainte, pas la conception de solutions IA. À travailler sérieusement, ou à assumer en proposant une phase de diagnostic longue.